

因子工程报告 (v3)

良文杯 2026 LOB 中间价方向预测——SchemeP 共 370 维因子拆解
统一 F1-F6 family 视角下的特征重要性 & NN 训练管线 NaN 处理

史景喆 (清华大学)

团队：超级回天什么时候上映 | 公榜 #1 (+35.64) | 私榜 #1

2026 年 5 月

1. 摘要

本报告对良文杯 2026 决赛冠军方案 SchemeP 的特征体系做完整技术性拆解。SchemeP 在 $T = 100$ tick 因果窗口上构造 370 维特征向量；按语义把全部 370 维划入六大家族 F1~F6 (互斥且完备)：F1 LOB 派生 (105 维)、F2 多尺度 OFI (50 维)、F3 订单流强度 (45 维)、F4 微结构波动 (29 维)、F5 窗口统计 (67 维)、F6 不对称性 (74 维)。

核心结论 (feature importance 的唯一 bucket = F1-F6 unified)：F1 占 LGB gain 40.4%、F3 占 35.1%，两者合计 75.5%，是预测主力；F4 (7.5%)、F6 (6.3%)、F2 (5.5%)、F5 (5.2%) 单家族 gain 较低但承担多尺度冗余与 OOD 稳健化。全特征单因子 gain 第一是 $ma_intst=0.945$ (隶属 F3)，Top 4 全部为 F3 * $_intst$ 。

Cross-cut：family 内部 raw vs derived 字段贡献 (370 维 = 154 维原始 + 216 维派生；这两段都被分入 F1-F6)：F1 的 96.5%、F3 的 82.6% gain 来自该家族内的原始字段 (LOB 10 档 + * $_intst$)，F6 raw 占 45.8%；F2/F4/F5 几乎完全靠该家族内的派生字段。这说明“派生工程的边际信息贡献”具有强烈的 family 异质性：F1/F3 的 raw 字段已是 LGB 主力，派生层在该家族内更多作为多窗口冗余；F2/F5 的派生层则是该家族的全部信号载体。在原始 154 维内部做更细粒度的 drilldown：order-flow intensity (* $_intst$) 6 维占 raw 段 gain 31.3%、LOB aggregate 4 维占 15.0%；midprice / spread / price_rate 40 维仅占 raw-gain 1.5% (近死特征)。

报告同时新增 §4 说明 NN 训练管线对 NaN 的处理：window-z 路径用 $np.nanmean / nanstd$ 算统计量、归一化后用 $nan_to_num(nan=0, posinf=10, neginf=-10) + clip(-10, 10)$ ，旁路 (无 window-z) 在 raw scale 直接 $nan_to_num(nan=0)$ ；LGB 路径完全不做 NaN 处理 (自带 missing 路由)。

2. 数据基底与原始 154 维字段

原始字段 (154 维)：来自盘口 snapshot parquet，每 tick 包含

- OHLC 与单 tick 增量：open, high, low, close, volume_delta, amount_delta (6 维)；
- 10 档买卖盘价 bid_k / ask_k (20 维) 与挂单量 $bsize_k / asize_k$ (20 维)， $k \in \{1..10\}$ ；
- 行情衍生量 $avgbid, avgask, totalbsize, totalasize$ (4 维)、 $bid_mean, ask_mean, bsize_mean, asize_mean$ (4 维)；
- 主办方提供的中位价与价差 $midprice_k, spread_k$ (20 维)、 $cumspread, imbalance$ (2 维)、跨档价差 bid_diff_k, ask_diff_k (20 维)；
- 六类订单事件强度——挂单 lb/la_intst 、成交 mb/ma_intst 、撤单 cb/ca_intst 及其 $_ind / _acc$ 变体 (18 维)；
- 主办方滑动平均变化率 $bid_rate_k / ask_rate_k / bsize_rate_k / asize_rate_k$ (40 维)。

派生因子 (216 维)：在长度 $T = 100$ 的滑动窗口上一次性向量化 (sliding_window_view) 后批量计算，按 stage 顺序拼接：

$$\underbrace{X_{raw}}_{154} \oplus \underbrace{X_{T3}}_{69} \oplus \underbrace{X_{S1}}_{54} \oplus \underbrace{X_{S2}}_{59} \oplus \underbrace{X_{S3}}_{14} \oplus \underbrace{X_{S5}}_{20} = 370.$$

T3 含 MLOFI / WMP / RV / EWMA-intst, Stage 1 含 dual-z / signed-RV / Kyle / EWMA-OFI, Stage 2 含 qrank / skew / GOFI / Kyle- λ / vol-burst, Stage 3 含 EWMA 残差 / RV ratio / jump-share / cancel-imb / Roll

效价比, Stage 5 含 adaptive-momentum / OFI-toxicity / signed-bipower / spread-regime / trade-persistence / liq-asm。

family **划分原则:** 全部 370 维被划入 6 个 互斥且完备的家族 F1-F6——这是本报告对特征重要性的唯一 bucket 视角。原始 154 维并不是一个独立家族, 而是分散到 F1 (94 维原始 LOB / OHLC)、F3 (18 维原始 *_intst/ind/acc)、F4 (2 维 volume/amount_delta)、F6 (40 维原始 *_rate)。" 原始 vs 派生" 以 cross-cut 形式出现在 §6.3 family 内部分解中。

标签: 5 个 horizon $h \in \{5, 10, 20, 40, 60\}$ 的 3 类方向 (涨 / 平 / 跌)。比赛主力使用 $h = 60$ 。

硬约束 (CRITICAL CONSTRAINTS): 评测时 date=0、sym 0-4 可含训练外股票、测试点顺序被打乱。因此所有 370 个因子都满足: (i) 不显式使用 date / sym; (ii) 每条样本独立可算, 不依赖跨样本状态; (iii) sym-agnostic, 不嵌入 sym embedding 或 sym-conditional 归一化。

3. F1-F6 派生因子六大家族

3.1 家族 F1: LOB 派生 (105 维)

动机。 盘口 snapshot 自身的价格水平、挂单深度与中位价是任何方向预测的最基本输入。本家族包含原始 OHLC、10 档买卖盘价/量、价差、中位价、加权微价格 WMP, 以及订单簿层间结构 bid_diff / ask_diff (同侧相邻档位间价格差)。它们刻画" 当下盘口长什么样", 是 F2-F6 衍生因子的基底。

因子清单。 OHLC 4 + bid k /ask k 20 + bsize k /asize k 20 + avgbid/avgask/totalbsize/totalasize 4 + bid_mean/ask_mean/bsize_mean/asize_mean 4 + midprice k 10 + spread k 10 + cumspread/imbalance 2 + bid_diff k /ask_diff k 20 + wmp_lv1 k 10 + wmp_balance_12 1 = 105 维。

核心代码——WMP (加权微价格):

```
for k in T3_LOB_LEVELS: # 1..10
    b = X3d[:, :, col_idx[f"bid{k}"]]
    a = X3d[:, :, col_idx[f"ask{k}"]]
    bs = X3d[:, :, col_idx[f"bsize{k}"]]
    asz = X3d[:, :, col_idx[f"asize{k}"]]
    denom = bs + asz
    wmp_normal = (a * bs + b * asz) / np.where(denom == 0, 1.0, denom)
    wmp_fallback = (a + b) / 2.0
    wmp = np.where(denom == 0, wmp_fallback, wmp_normal)
    out[:, col] = wmp[:, -1]; col += 1
out[:, col] = wmp_per_level[0][:, -1] - wmp_per_level[1][:, -1] # wmp_balance_12
```

WMP 分子用 ask*bsize + bid*asize (" 短缺侧权重重大"), 符合 Stoikov 微价格直觉, 挂单越少的一侧把 WMP 拉得越接近其报价。

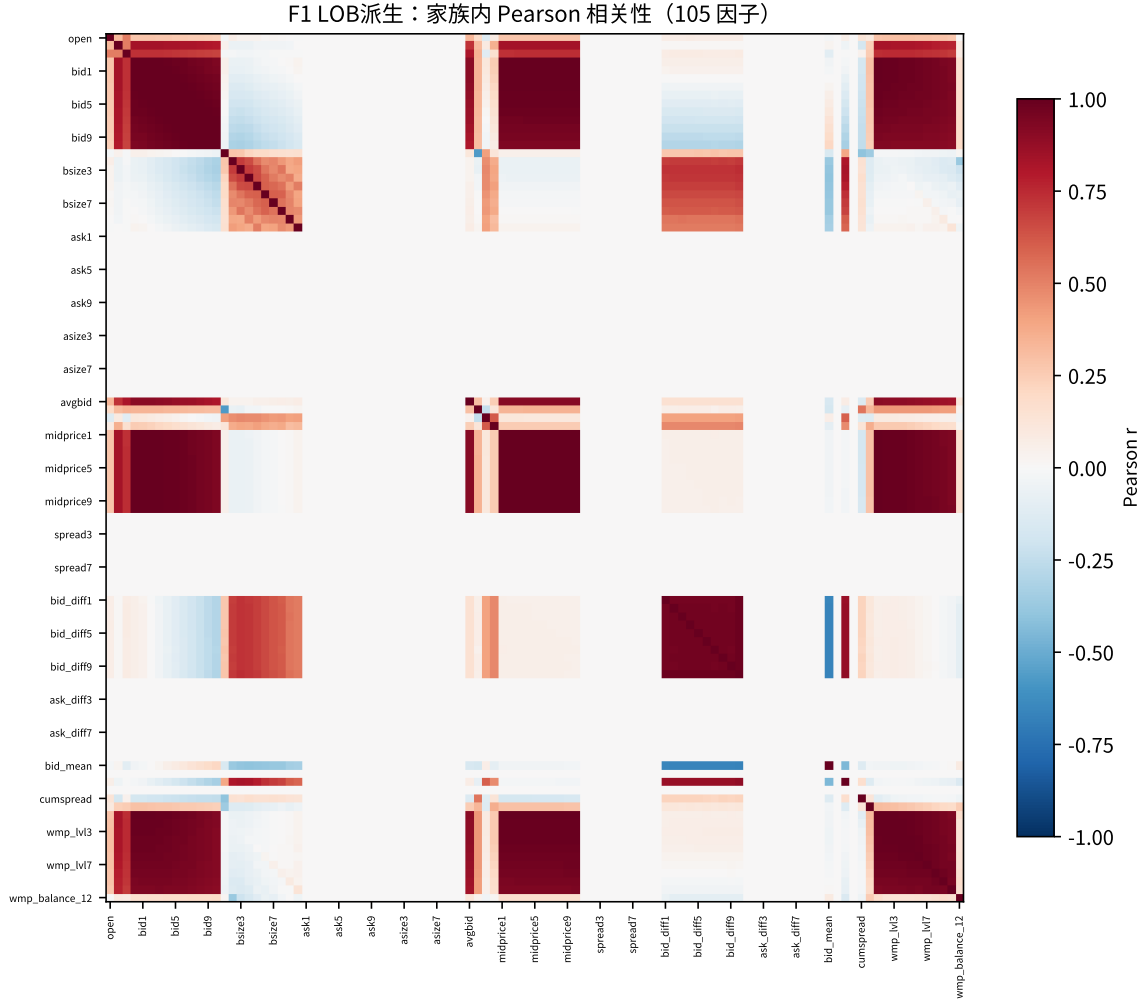


Figure 1: F1 家族内 Pearson 相关性 (采样 10 万行 train)，平均 $|r| = 0.154$ 。10 档同名因子 (如 bid1..bid10) 形成明显对角块；midprice k 与 wmp_lv k 高度相关 (同向价值)。

3.2 家族 F2：多尺度 OFI (50 维)

动机。 订单流不平衡 (OFI, Cont 2014) 是预测短期方向的最强信号家族。SchemeP 在多窗口 ($W \in \{5, 20, 60\}$ for MLOFI; $W \in \{50, 100\}$ for Kyle/toxicity) 与多 EWMA 半衰期 ($\alpha \in \{0.05, 0.1, 0.3, 0.5\}$) 乘积空间内构造 50 维 OFI 表达。本家族 100% 派生，无原始字段。

因子清单。 $\text{omlofi_WW_lvl}k \ 30 + \text{ewma_ofi_a}\alpha_lvlk \ 12 + \text{kyle_inv_WW} \ 2 + \text{kyle_lam_WW} \ 2 + \text{ofi_tox_lvl}k_WW \ 4 = 50$ 维。

核心公式——MLOFI (Cont 2014 multi-level OFI)： 对每一档 k ,

$$e_t^{(k)} = \mathbf{1}_{b_t^{(k)} \geq b_{t-1}^{(k)}} bs_t^{(k)} - \mathbf{1}_{b_t^{(k)} \leq b_{t-1}^{(k)}} bs_{t-1}^{(k)} - \mathbf{1}_{a_t^{(k)} \leq a_{t-1}^{(k)}} as_t^{(k)} + \mathbf{1}_{a_t^{(k)} \geq a_{t-1}^{(k)}} as_{t-1}^{(k)}.$$

再做窗口求和 $\text{MLOFI}_{t,W}^{(k)} = \sum_{s=t-W+1}^t e_s^{(k)}$ 。 b 上行 (或持平) 时贡献 $+bs$; b 下行时减去前一档 bs (即被吃单或撤单消失的挂量)，左右对称构成净 bid pressure。

核心公式——反 Kyle 冲击： $\text{KyleInv}_W = \frac{\Delta \text{amt}}{\sqrt[3]{|\Delta \text{amt}| \cdot \sigma(r_W) + \varepsilon}}$ ，直觉是 "在活跃度下，净流动金额能解释多少方向"。

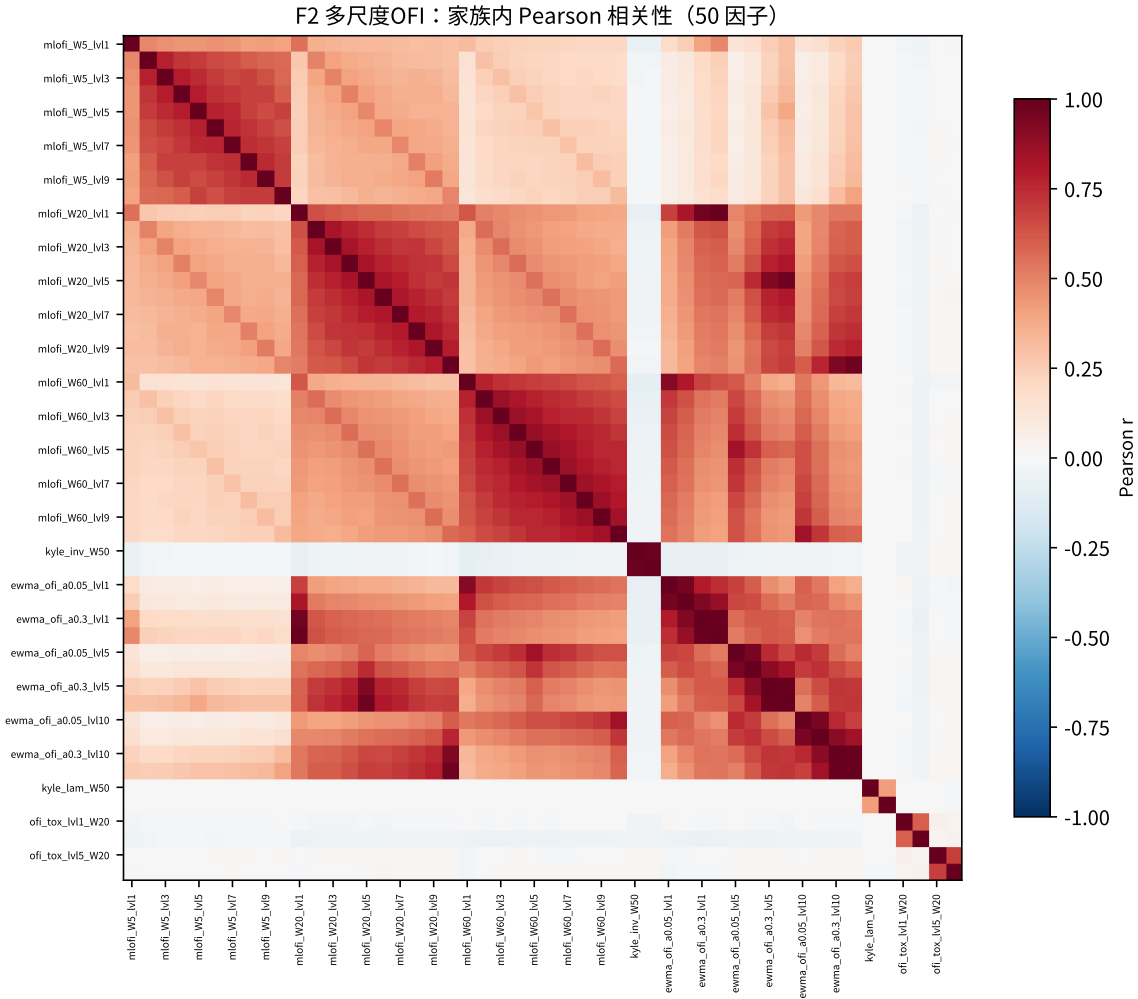


Figure 2: F2 家族内 Pearson 相关性, 平均 $|r| = 0.33$ (六族之最)。多尺度 OFI 内部高度共线：同一 EWMA- α 下不同 level 的 OFI、同一 level 下不同 W 的 MLOFI 都几乎平行。

3.3 家族 F3：订单流强度 (45 维)

动机。比赛数据有别于传统 LOB 任务的关键在于：主办方直接给出了 6 类订单事件的强度，limit / market / cancel 三类 $\mathbb{C}$ buy/sell 两侧 = 6 列，并配 $_ind$ / $_acc$ 变体。家族 45 维 = 原始 18 维 + 派生 27 维；LightGBM 给本家族 35.1% 的 gain。Top 4 全局重要性都在 F3，分别是 ma_intst 0.945、 mb_intst 0.860、 la_intst 0.650、 lb_intst 0.567，全部为原始字段。

因子清单。 $lb/la/mb/ma/cb/ca_intst$ 6 + $_ind$ 6 + $_acc$ 6 + $ewma_a\alpha_c_intst$ 24 + $cancel_imb_WW$ 3 = 45 维。

核心公式——EWMA-滤波订单强度： $\hat{x}_t^{(\alpha)} = \alpha x_t + (1 - \alpha) \hat{x}_{t-1}^{(\alpha)}$ 。通过 `scipy.signal.lfilter` 的 IIR 形式 ($b = [\alpha], a = [1, -(1 - \alpha)]$) 批量计算。

Cancel imbalance:

$$cancel_imb_W = \frac{\sum cb}{\sum(lb + mb + cb) + \varepsilon} - \frac{\sum ca}{\sum(la + ma + ca) + \varepsilon}.$$

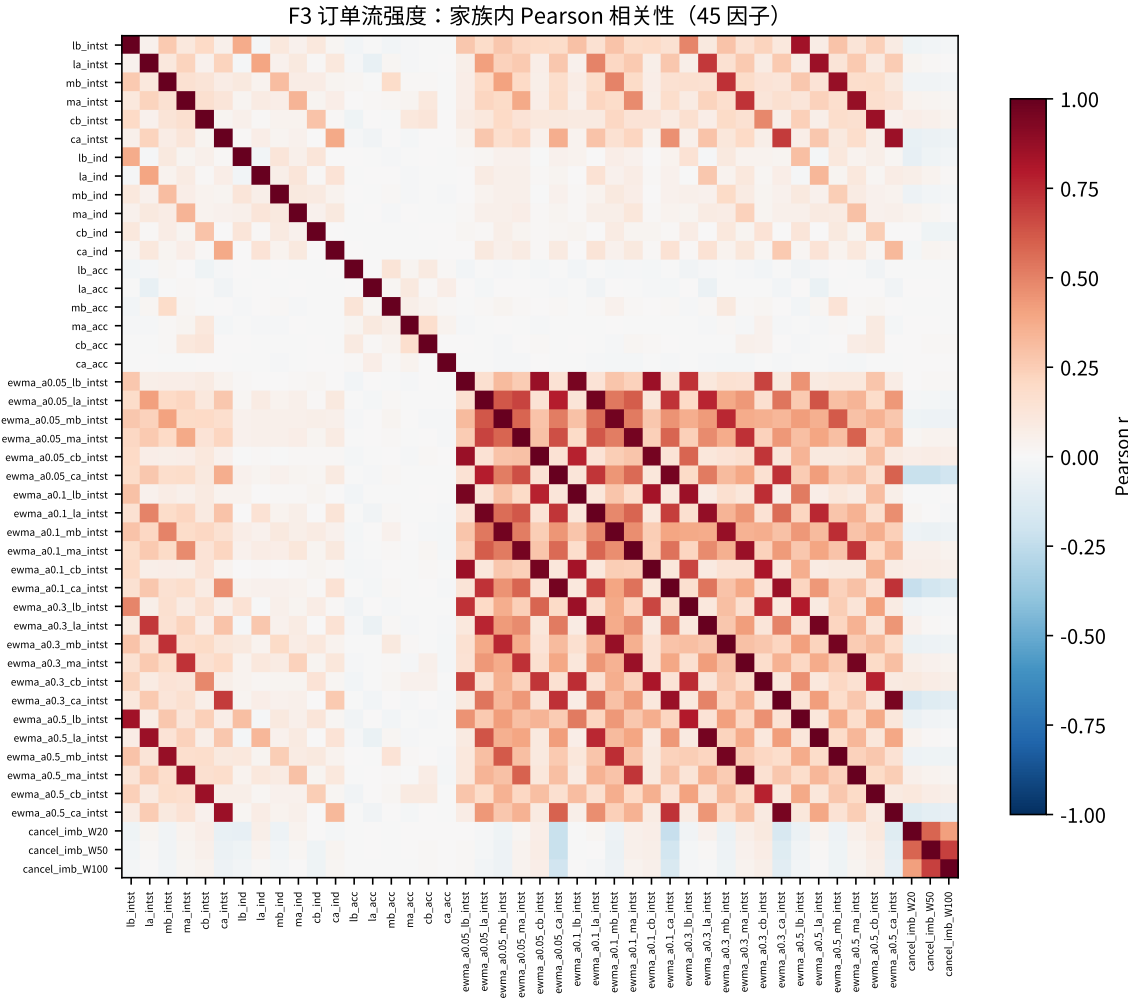


Figure 3: F3 家族内 Pearson 相关性，平均 $|r| = 0.18$ 。6 类原始强度 $\times$ 4 个 EWMA- α 形成 24 维 EWMA 块；cancel_imb_W* 三档跨窗口高度相关但与挂单/成交强度近乎正交。

3.4 家族 F4：微结构波动 (29 维)

动机。波动率不仅决定信号强度，也决定阈值与仓位。SchemeP 用 4 个角度刻画微结构波动：无符号波动率 $RV = \sum r^2$ 、方向化波动率 signed RV、跳跃稳健估计二次方差 (bipower BV)、以及 活跃度突发 vol/amt burst。

因子清单。volume_delta/amount_delta 2 + rv_wW 4 + signed_rv_WW 3 + rskew_WW 3 + signed_bv_WW 3 + vol/amt_burst_WW 4 + rv_ratio 3 + jshare_WW 4 + roll_eff_spr_ratio_WW 3 = 29 维。

核心公式：signed RV = $(RV_+ - RV_-)/(RV_+ + RV_-) \in [-1, 1]$; jump share = $\max(0, (RV_W - BV_W)/RV_W)$, $BV_W = \frac{\pi}{2} \sum |r_t| |r_{t-1}|$ (价格连续时 $BV \approx RV$, 跳跃时 $RV \gg BV$); Roll 有效价差 = $2\sqrt{\max(0, -\text{cov}(\Delta P_t, \Delta P_{t-1}))}$ 。

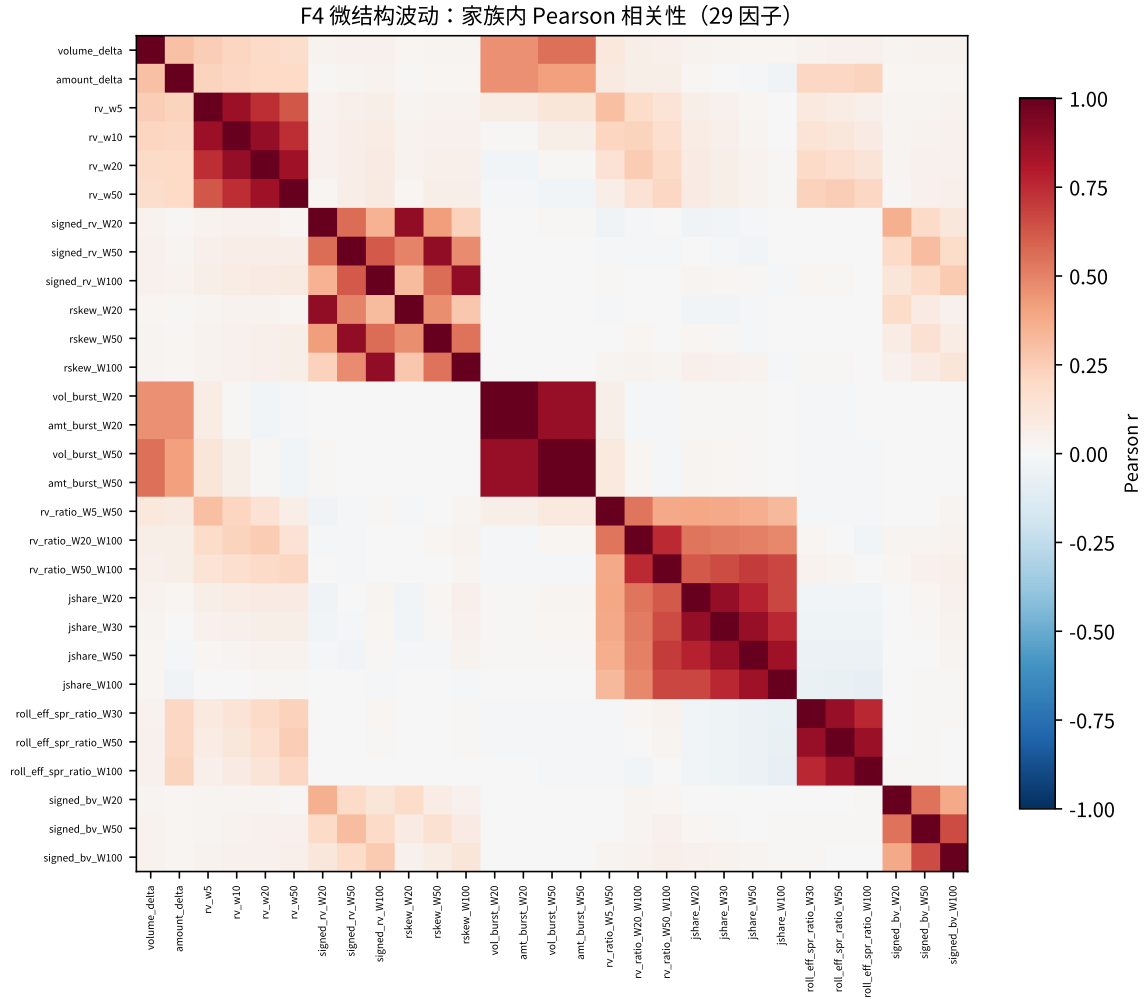


Figure 4: F4 家族内 Pearson 相关性，平均 $|r| = 0.16$ 。rv_w* 四个窗口高度相关（无符号波动率同源），jshare / signed_rv 与 rv 几乎正交。

3.5 家族 F5：窗口统计 (67 维)

动机。本家族是 SchemeP 区别于纯" 原始 LOB + OFI" 基线的胜负手，也是解决 OOD 漂移的核心。所有 67 维全为派生 (family-row=0)，把" 绝对水平" 通过 100-tick 窗口内的统计量 (z-score、quantile rank、median-IQR、长短期 z-score 差) 转成" 相对水平"。

因子清单: dualz_c 37 + qrank_W100_c 20 + mid_ewma_resid 1 + adapt_mom_WW 3 + spread_reg_WW 3 + trade_pers_WW 3 = 67 维。

核心公式——Dual-window z-score (决胜 OOD 的"trick"):

$$\text{dualz}(c)_t = \frac{c_t - \mu_{20}(c)}{\sigma_{20}(c) + \varepsilon} - \frac{c_t - \mu_{100}(c)}{\sigma_{100}(c) + \varepsilon}.$$

等价于" 当前 tick 相对短期均值偏离 多于长期均值偏离的程度"，对均值水平不敏感（两个 z 相减消除全局基差），对方差也鲁棒（除掉 σ ）。SchemeP 对 37 个 base 列做 dualz。

```
dz_idx = np.array([col_idx[c] for c in DUAL_Z_COLS])
X_dz = X3d[:, :, dz_idx] # (N, T, 37)
last = X_dz[:, -1, :]; seg20 = X_dz[:, -20:, :]; seg100 = X_dz[:, -100:, :]
m20, s20 = seg20.mean(1), seg20.std(1, ddof=0)
m100, s100 = seg100.mean(1), seg100.std(1, ddof=0)
z_s = (last - m20) / (s20 + EPS); z_l = (last - m100) / (s100 + EPS)
dual = np.where(np.isfinite(z_s - z_l), z_s - z_l, 0.0) # (N, 37)
```

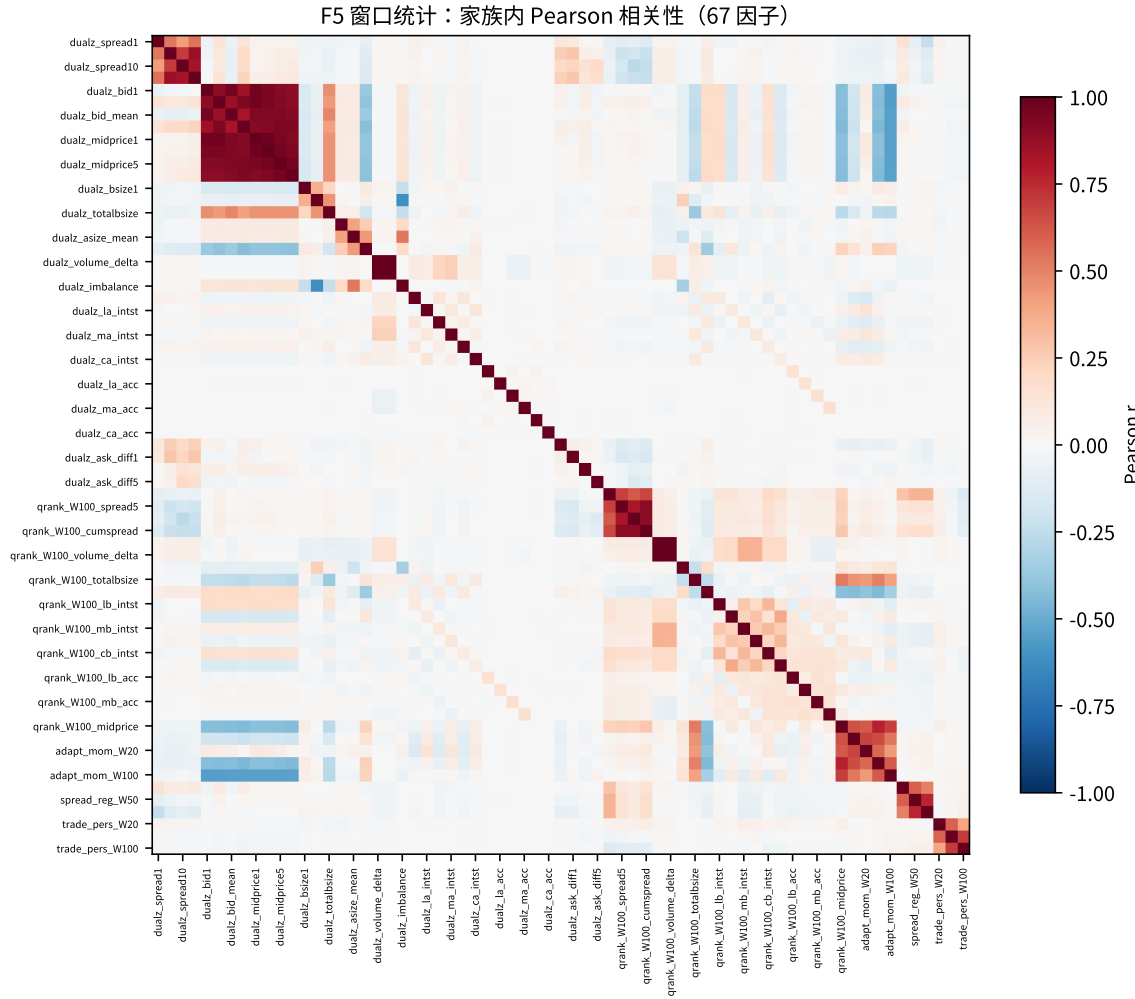


Figure 5: F5 家族内 Pearson 相关性, 平均 $|r| = 0.07$ (六族之最低)。Dual-z 因子之间几乎相互正交, 是 SchemeP 信息密度最高的家族。

3.6 家族 F6: 不对称性 (74 维)

动机。本家族刻画 买卖两侧的相对差异: 广义 OFI (GOFI) 量化" 价格档位变化下挂量净流向", liq-asym 用 top-5 size 比值取对数; *_rate 用滚动平均归一化每档价/量。家族 74 维 = 原始 40 维 (*_rate) + 派生 34 维。

因子清单:gofi_WW_lv1k 30 + liq_asym_top5_WW 4 + bid_ratek/ask_ratek 20 + bsize_ratek/asize_ratek 20 = 74 维。

核心公式——GOFI: 区别于 MLOFI 严格按" 价格上下行 + 同档量增减", GOFI 把同价不变时的挂量净变化也算进去:

$$e_t^{(k)} = (\mathbf{1}_{b\uparrow}bs_t - \mathbf{1}_{b\downarrow}bs_{t-1} + \mathbf{1}_{b=}(bs_t - bs_{t-1})) + (\mathbf{1}_{a\uparrow}as_{t-1} - \mathbf{1}_{a\downarrow}as_t - \mathbf{1}_{a=}(as_t - as_{t-1})).$$

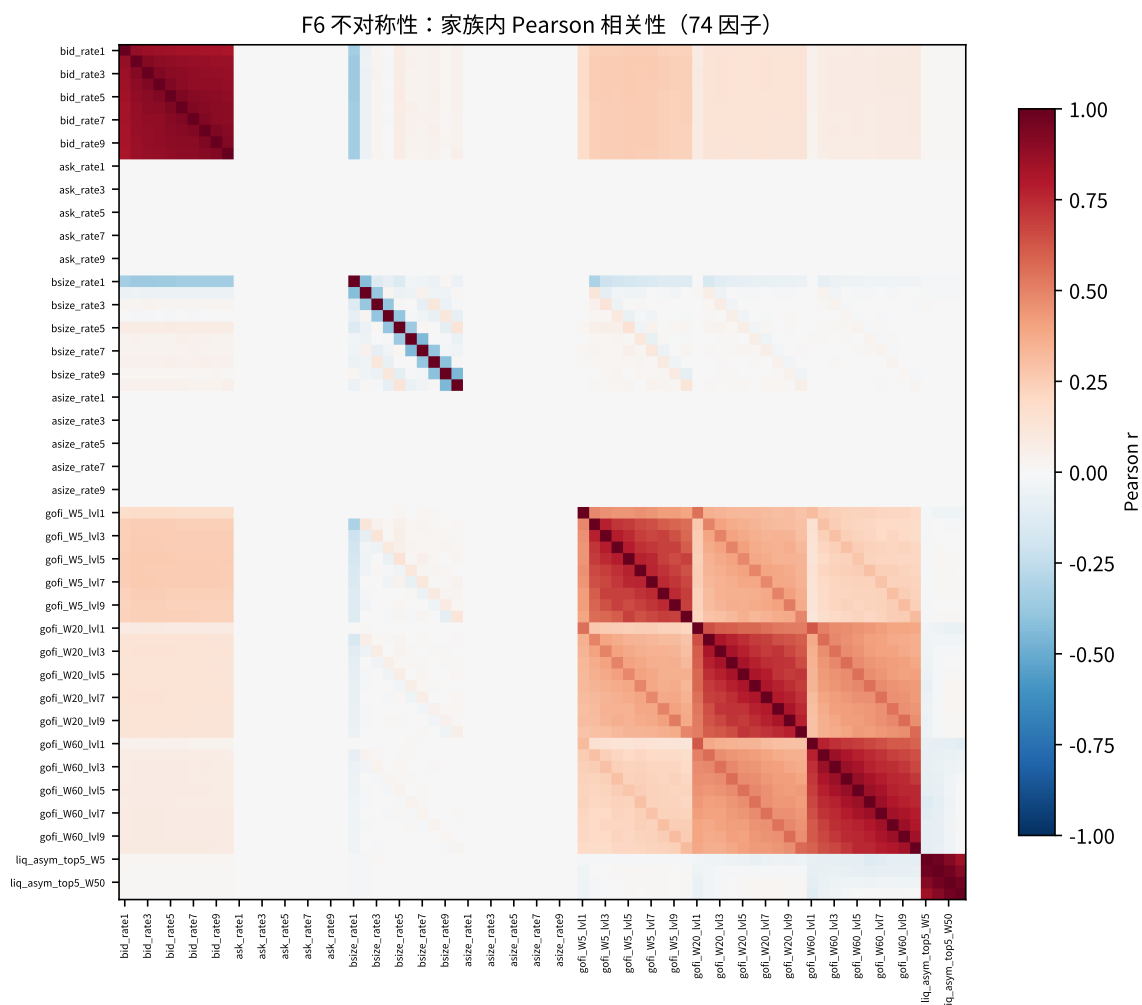


Figure 6: F6 家族内 Pearson 相关性, 平均 $|r| = 0.12$ 。GOFI 30 因子块跨档/跨窗口相关较强; *_rate 4 个子块彼此正交, 但因子块内 10 档内高度相关。

4. NN 训练管线的 NaN 处理

NN 与 LGB 的关键差异之一是: NN 不能容忍 NaN/Inf, 否则前向出 nan loss、反向梯度爆炸; 而 LGB 自带 missing value 路由 (每个 split 学一个 NaN-direction)。SchemeP 的 NN 管线对 NaN 做严格的"两层兜底"。

4.1 数据集本身的 NaN 分布

完整 SchemeP cache (ablation_runs/cache_log1p) 在 154 维原始字段上扫描:

- 1192 / 1200 个文件无 NaN (绝大多数 tick 的盘口 10 档都填满);
- 8 个文件 (涨停日 / 跌停日) 深档 ask 或 bid 出现少量缺失——盘口被打穿、深档无报价;
- 派生 216 维由 sliding_window_view + 向量化算式产出, 所有 NaN 已在派生公式里用 np.where (denom==0, fallback, normal) 兜过一次 (如 WMP fallback 到普通 mid, dualz 用 np.isfinite 兜底);
- 因此进入 NN 管线时, 预期 NaN 数 $\ll 10^{-5}$ 占比, 但不能假设没有——必须设最后一层兜底。

4.2 主路径: window-z 开启时的 NaN 处理

代码位置 ablation_runs/rerun_13rows.py:271-285:

```
# 5. window-z (global mu/sd on train)
if cfg["wz"]:
    mu = np.nanmean(tr["X"], axis=0).astype(np.float32)
    sd = np.maximum(np.nanstd(tr["X"], axis=0),
                    1e-6).astype(np.float32)

    def zsc(X):
        out = (X - mu) / sd
        np.nan_to_num(out, copy=False,

# (1) nan-aware mean
# (1) nan-aware std
# (2) clamp sd >= 1e-6
# (3) standardize
```



```

nan=0.0, posinf=10.0, neginf=-10.0) # (4) NaN->0, +-inf->+-10
np.clip(out, -CLIP, CLIP, out=out) # (5) clip to [-10, 10]
return out.astype(np.float32)
tr["X"] = zsc(tr["X"]); va["X"] = zsc(va["X"]); te["X"] = zsc(te["X"])

```

五步策略与理由：

1. `np.nanmean` / `nanstd` 算 train 全局统计量——若用普通 `mean`，任意一行有 NaN 都会污染整列均值；`nan-aware` 版本只忽略缺失格子。
2. `sd clamp` 到 $\geq 10^{-6}$ ——防止常数列（如某 `sym` 在某窗口 `spread=const`）导致 0 除。
3. 用 train 的 μ, σ 直接套到 val/test——保证三集统计同口径，避免 train/val/test 分布偏移。
4. `nan_to_num(nan=0)`：归一化之后 NaN 已是“未知偏离度”，置 0 = “等于均值” = 最保守的 `mean-imputation`。
`posinf=10, neginf=-10`：极端值映射到 ± 10 （即 $\pm \text{CLIP}$ ），与下一步 `clip` 兜底一致。
5. `clip(-10, 10)`：极端 outlier（如 `sigma=0` 时被除爆的样本）压回边界，避免梯度爆炸。`CLIP=10` 来源于 `rerun_13rows.py:49`。

4.3 旁路：无 window-z 的 raw-scale NaN 处理

代码 `rerun_13rows.py:283-288`：

```

else:
    for arr in (tr["X"], va["X"], te["X"]):
        np.nan_to_num(arr, copy=False, nan=0.0) # raw scale: NaN -> 0
    mu = np.zeros(tr["X"].shape[1], dtype=np.float32)
    sd = np.ones(tr["X"].shape[1], dtype=np.float32)

```

这是 保守旁路：raw scale 下 `NaN` $\rightarrow 0$ 在数值上未必“等于均值”（因为 0 不是经验均值），但对极少数 NaN 的样本来说，仍然是“该位特征贡献 0”——比让前向出 NaN 强。这条路径主要用在控制实验（消融 `window-z` 看效果），不是主提交。

4.4 与 LGB 路径的对比

模型	NaN 处理	理由
LGB	完全不动	LightGBM 每个 split 学一个 NaN-direction，把缺失当第三个 bin：对该 feat = NaN 的样本，整体路由到左/右子树之一。比手动 <code>imputation</code> 信息量更高。
NN (主路径)	<code>nanmean</code> / <code>nanstd</code> $\rightarrow$ <code>zsc</code> $\rightarrow$ <code>nan_to_num(0)</code> $\rightarrow$ <code>clip</code>	NN 前向不能容忍 NaN；归一化后 <code>NaN</code> $\rightarrow 0$ = “等于 train 均值”，保守。
NN (旁路 raw)	<code>nan_to_num(0)</code> 直接在 raw scale	不等于均值 ($0 \neq \mu$)，但仍能保证前向有限值。控制实验用。

Table 1: LGB 与 NN 两条管线对 NaN 的处理对比。

关键工程教训：`np.nanmean` 与 `np.mean` 一字之差，决定了“是否被 NaN 污染全列”。SchemeP 早期版本曾因为漏掉 `nan*` 前缀，导致某一列均值变成 `NaN` $\rightarrow$ 全部样本归一化后变成 `NaN` $\rightarrow$ 训练崩溃。这是 NN-on-LOB 工程里最容易 `misstep` 的一行代码。

5. 三类关键预处理

5.1 Sign-preserving log1p（重尾压缩）

针对 `amount_delta` 这种 6σ 重尾且可正可负的字段： $x \mapsto \text{sign}(x) \cdot \log(1 + |x|)$ 。保留方向，将量纲从 $\sim 10^8$ 压缩到 ~ 18 。

```

raw_last = X3d[:, -1, :].astype(np.float32)
amt_idx = col_idx["amount_delta"]
v = raw_last[:, amt_idx]
raw_last[:, amt_idx] = np.sign(v) * np.log1p(np.abs(v))

```

5.2 Window-z (跨股票/跨日基差消除)

详见 g3.5 dualz 实现。**为什么对 OOD 关键：**LightGBM 分裂阈值是绝对值；若 train 一只股票 spread1 $\sim 10^{-3}$ ，test OOD 股票 spread1 $\sim 3\times 10^{-3}$ ，则训练阈值在测试上全错位。Window-z 把绝对值映射到无量纲 z，"分裂可比性" 得到恢复。

公榜 OOD 阶段 PnL 从 -6.65 跃升至 $+35.64$ (增益 $+42.29$)，window-z 是其中两个关键 trick 之一。

5.3 Mirror augmentation (镜像增强)

每条 $(X, y_{\text{reg}}, y_{\text{cls}})$ 复制一份并 镜像：所有 "bid 类" 与对应 "ask 类" 列对换、 $y_{\text{reg}} \rightarrow -y_{\text{reg}}$ 、 $y_{\text{cls}} \rightarrow 2 - y_{\text{cls}}$ 。强制模型学到完美的 bid/ask 对称性，对 50-seed bagging robust 起到 "先验对称化" 作用。

6. 特征重要性的统一分析

本节使用 **统一的 F1-F6 family 视角** 分析 LightGBM 的特征重要性。370 维全部归入这 6 个互斥家族，因此 "家族 gain 占比" 是完备的 (六者之和 = 100%)。"原始 vs 派生" 不作为同层级 bucket (它们本来就是 F1-F6 的子集)，而是以 cross-cut 形式在 g6.3 出现。

模型选自 big50_lgb_m7/seed=1 (M7 重训第 1 个种子，370 维全量)，单 seed gain 与 50-seed 平均 gain rank-corr > 0.95 。

6.1 家族级 gain 分解 (F1-F6 unified)

家族	因子数	占总数 (%)	LGB gain 占比 (%)
F1 LOB 派生	105	28.4	40.4
F2 多尺度 OFI	50	13.5	5.5
F3 订单流强度	45	12.2	35.1
F4 微结构波动	29	7.8	7.5
F5 窗口统计	67	18.1	5.2
F6 不对称性	74	20.0	6.3
合计	370	100	100

Table 2: 六大家族因子数与 LightGBM 特征重要性 (gain, h=60 M7 模型 seed=1)。F1 + F3 = 75.5%。

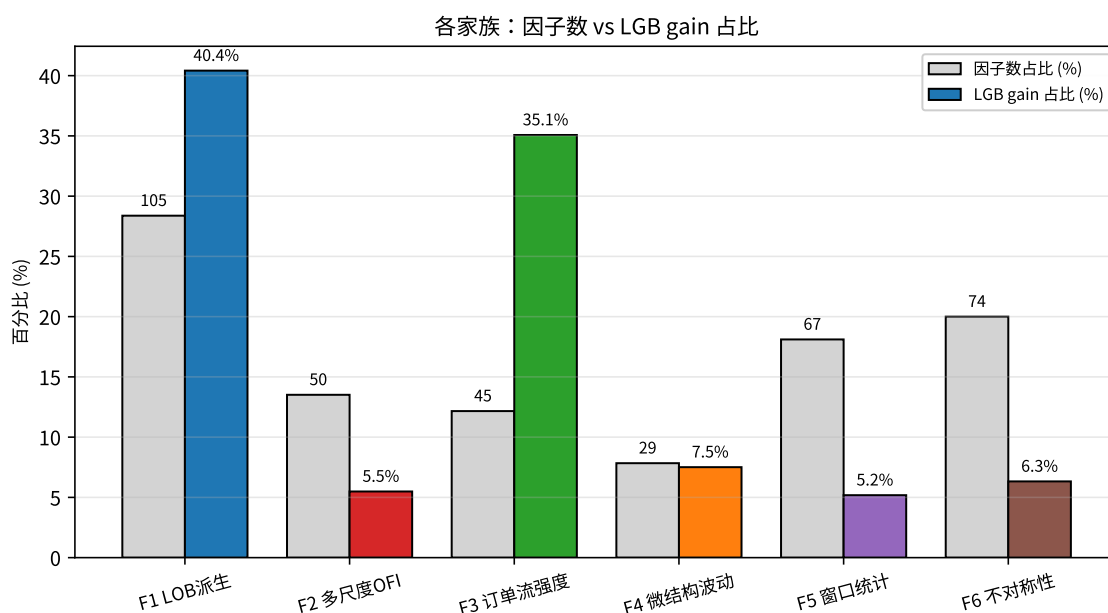


Figure 7: 各家族因子数与 gain 占比对比。F1 (28.4% 因子 / 40.4% gain) 与 F3 (12.2% 因子 / 35.1% gain) 单因子贡献最强；F5 (18.1% / 5.2%) 单因子最弱但近正交 (族内 $|r| = 0.07$)，承担 OOD 稳健化。

单因子相对贡献：定义 = 家族平均单因子 gain / 全局平均单因子 gain。F3 = $2.89\times$ (最高)、F1 = $1.42\times$ ；F5 = $0.29\times$ 、F6 = $0.32\times$ 、F2 = $0.41\times$ 。F3 与 F5 差近 10 倍——LightGBM 一颗树一次只看一个特征做分

裂，强单因子（F3 原始 *_intst）有压倒性优势；而 F5 的 z-score 因子" 个个不显眼，胜在彼此正交，组合起来形成稳健的决策边界"。单因子重要性低 $\neq$ 家族不重要。

6.2 全局 Top 40 因子

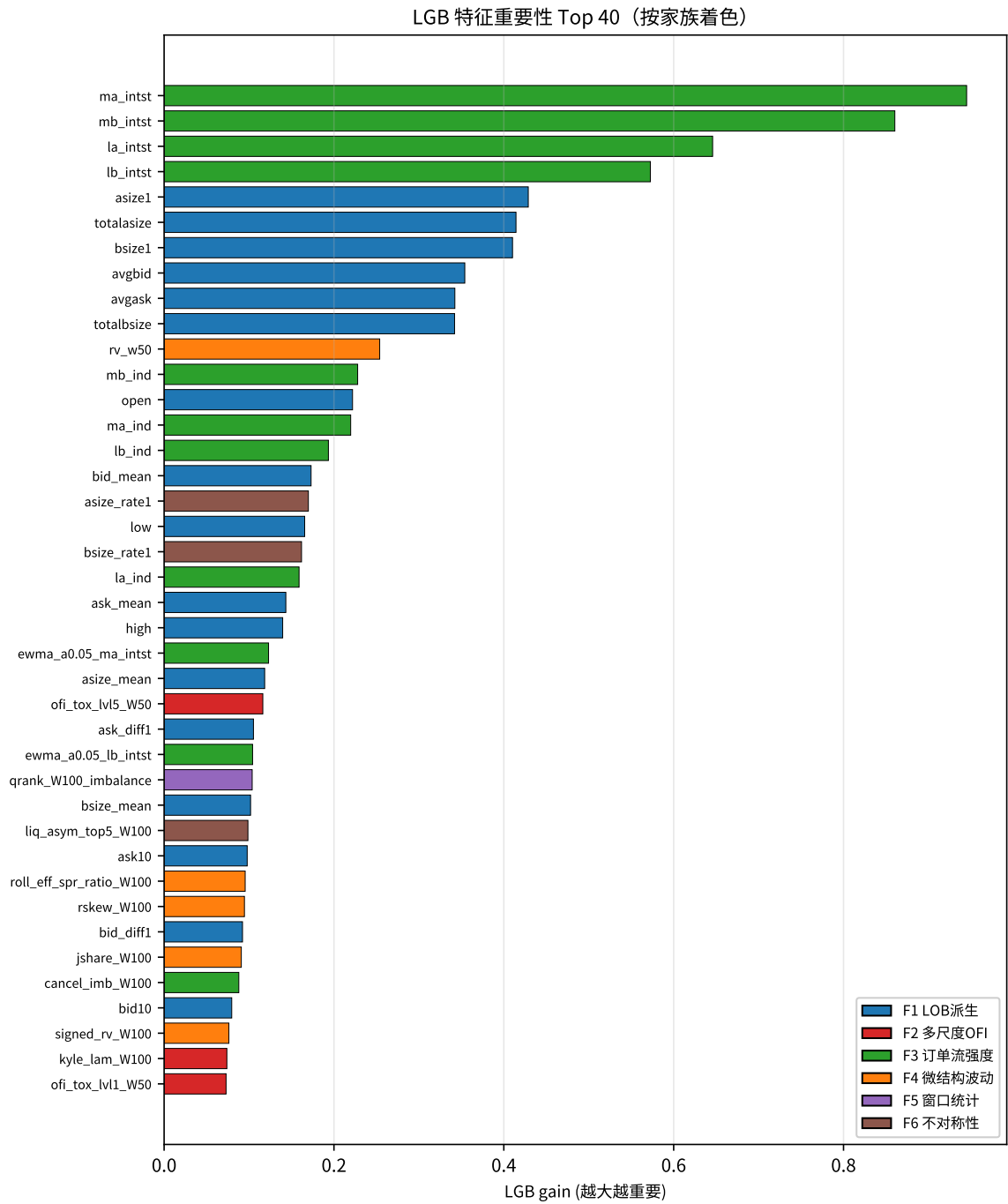


Figure 8: LGB feature importance Top 40 (按家族着色)。Top 4 全部为 F3 ma/mb/la/lb_intst；紧接着 F1 的 asize1, totalasize, bsize1, avgbid/avgask, totalbsize, asize2。ma_intst gain = 0.945 居首，mb_intst 0.860 第二。Top 20 几乎全部隶属 F1 或 F3。

6.3 Cross-cut：家族内 raw vs derived 字段贡献

将 370 维按 F1–F6 family $\times$ 是否在原始 154 维内做 2-way 分解，能回答： " 每个家族内部，raw 字段和 derived 字段各占多少 gain? "

家族	n_{raw}	gain_{raw}	raw %	n_{der}	gain_{der}	der %	family share %
F1 LOB 派生	94	5.327	96.5	11	0.196	3.5	40.42
F2 多尺度 OFI	0	0.000	0.0	50	0.750	100.0	5.49
F3 订单流强度	18	3.958	82.6	27	0.835	17.4	35.08
F4 微结构波动	2	0.006	0.6	27	1.020	99.4	7.51
F5 窗口统计	0	0.000	0.0	67	0.708	100.0	5.18
F6 不对称性	40	0.396	45.8	34	0.468	54.2	6.33
合计	154	9.687	70.9	216	3.979	29.1	100.00

Table 3: F1-F6 内部 raw / derived 字段贡献分解。raw 字段属于"前 154 维原始字段被划入该家族的部分", derived 字段属于"该家族中由滑动窗口公式派生出来的部分"。percent 列以同家族内总 gain 为分母 (行内 raw% + der% = 100)。最后一行的 70.9% / 29.1% 仅作合计参考, 不是与 F1-F6 同层的 bucket。

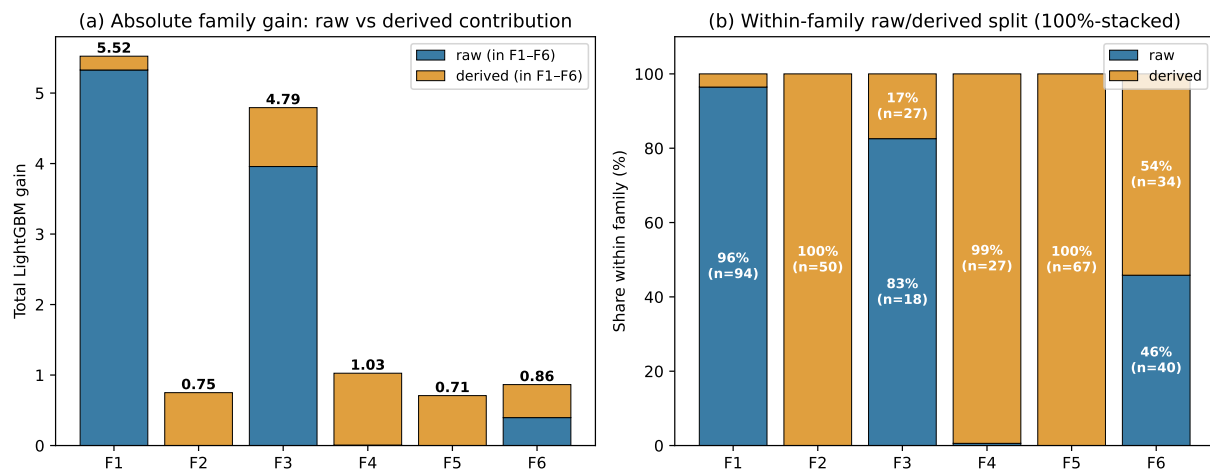


Figure 9: 家族内 raw vs derived 贡献: (a) 绝对 LGB gain 堆叠柱, 柱顶标注家族总 gain; (b) 同一柱按 100% 标定, 柱内标注百分比与维度数。F1/F3 的 raw 字段占绝对主导 (96.5%/82.6%); F2/F4/F5 几乎完全靠 derived; F6 接近五五分。

关键观察:

1. F1 的 raw 占 96.5%——LOB 10 档原始价/量、聚合量、OHLC 等 94 维 raw 字段已是 F1 的全部 gain 来源, 11 维派生 (主要是 WMP) 只补 3.5%。
2. F3 的 raw 占 82.6%——6 维 *_intst 几乎独占 F3, 27 维 EWMA 滤波派生只补 17.4%。
3. F5 的 derived 占 100%——dualz / qrank / mid-ewma-resid 全部是窗口统计派生, 本就不可能"raw"。F5 的 5.2% gain 是无任何 raw 字段帮助拿到的。
4. F6 接近五五分 (raw 45.8% / der 54.2%) ——40 维 *_rate 是 raw 字段, 34 维 GOFI / liq_asym 是 derived; GOFI 与 *_rate 各占一半。

诠释: 派生工程的"边际信息贡献"具有强烈的 family 异质性。F1/F3 内派生层属"多窗口冗余"; F2/F4/F5 内派生层是 该家族的全部信号载体, 没有派生就没有这些家族。这条 cross-cut 与 g6.1 的家族级 gain 分解互不冲突——前者答"家族内 raw vs derived", 后者答"家族整体强弱"。

6.4 细粒度: raw 154 维内的字段类型分解

把原始 154 维按字段语义拆成 17 个类型 (OHLC、bid 价、ask 价、bid 量、ask 量、LOB 聚合、midprice、spread、bid_diff、ask_diff、LOB simple aggregates、价格 rate、量 rate、order-flow intst/ind/acc 三变体), 统计每个类型在 raw 154 维内部的 gain 占比与单因子 gain。注意: 本表的 share 列分母为 raw-154 段内部总 gain (即 9.687), 不与 F1-F6 同口径。

类型	#feat	sum gain	share raw (%)	avg per feat
order flow intensity (intst)	6	3.032	31.30	0.505
LOB aggregate (avgbid/avgask/total*size)	4	1.453	15.00	0.363
order flow indicator (ind)	6	0.801	8.27	0.133
ask size L1-L10	10	0.699	7.22	0.070
bid size L1-L10	10	0.662	6.83	0.066
LOB simple aggregates (bid_mean/imbalance/cumspread/...)	6	0.595	6.14	0.099
OHLC	4	0.535	5.52	0.134
ask price L1-L10	10	0.382	3.95	0.038
bid price L1-L10	10	0.372	3.84	0.037
size rate (bsize/asize_rate1-10)	20	0.350	3.62	0.018
ask_diff L1-L10	10	0.286	2.95	0.029
bid_diff L1-L10	10	0.246	2.54	0.025
order flow acceleration (acc)	6	0.125	1.29	0.021
spread levels	10	0.059	0.61	0.006
price rate (bid/ask_rate1-10)	20	0.046	0.47	0.002
midprice levels	10	0.039	0.40	0.004
volume / amount delta	2	0.006	0.06	0.003
合计 raw 154	154	9.687	100.00	0.063

Table 4: 原始 154 维按字段语义分组后的 LGB gain 占比与单因子均值。share 列以 raw-154 段内部总 gain 为分母（即 9.687），不与 F1-F6 family share 同口径。

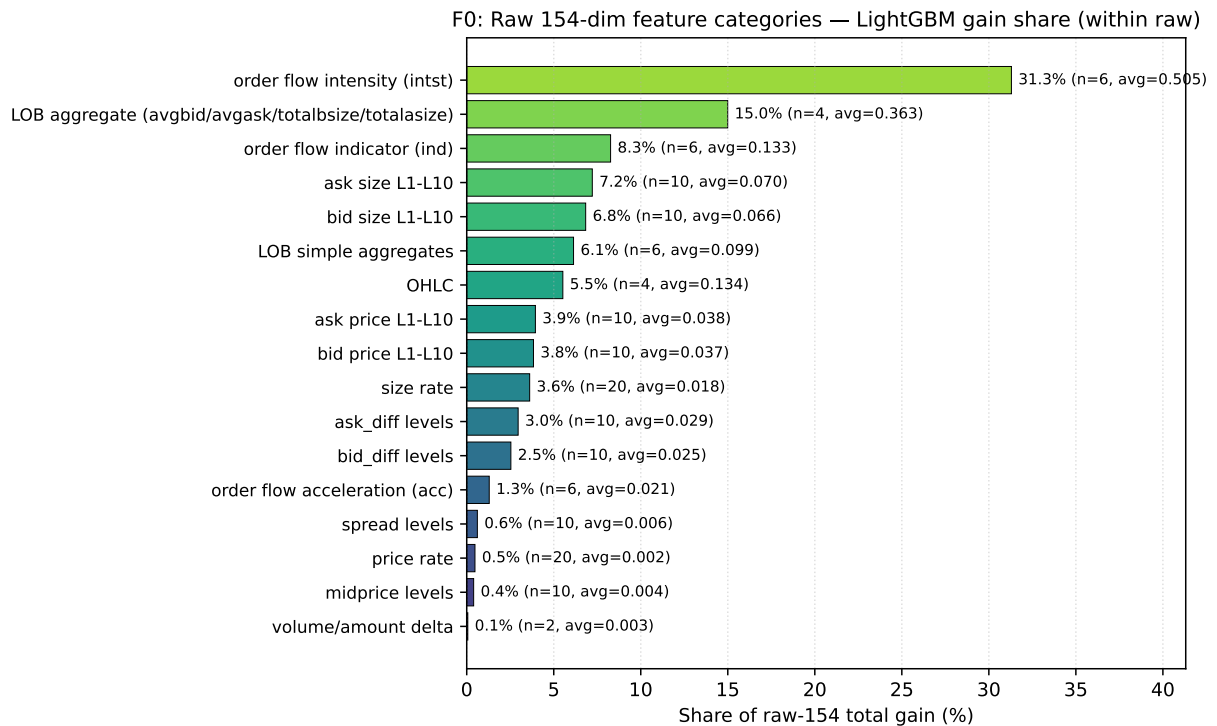


Figure 10: 原始 154 维内部按 17 个字段类型的 LGB gain 占比（以 raw-154 段总 gain = 100% 标定）。右侧标注 占比、维度数、平均单因子 gain。intst 6 维一骑绝尘（31.3%）；最末三类（midprice、price rate、volume/amount delta）几乎是死特征。

关键观察（raw 154 维内部）：

1. Order flow intensity (*_intst) 6 维占 raw-段 gain 31.3%：单因子最强 (ma_intst=0.945、mb_intst=0.860)，是已脱敏的撮合事件计数，比"由价量反推 OFI"直接得多。这 6 维属 F3 family。
2. LOB aggregate (4 维) 比 10 档原始价格更有用：4 维占 raw-段 15.0%、平均单因子 gain 0.363，远高于 10 档 ask price (3.95%、0.038/因子)。原因：聚合量 totalasize / totalbsize / avgbid / avgask 已把 10 档信息压缩到 1 维有效信号，避免了"10 档分散重要性"。这 4 维属 F1 family。
3. midprice / spread / price_rate 几乎死特征（每 feat < 0.01）：合计 40 维但只贡献 raw-段 1.5%。原因：

midprice 严重共线 (10 档 midprice 之间 $|r| > 0.99$)，spread 在标准化前是绝对值 (受 sym 基差污染)，price_rate 已被 LOB derivative 主导。

4. volume / amount delta **几乎为 0** (0.06% raw-段 gain)：原始单 tick 净流量信号已被 *_intst 完整接管——后者是更精细的"按事件类型拆分"版本。
5. ind/acc **变体相比 intst 大幅退化**：6 类 *_ind 合计 8.27% raw-段 gain，*_acc 仅 1.29%。intst 的"强度"比"指示"或"加速度"信息更丰富。

6.5 Order-flow 6×3 子矩阵

进一步把 18 维 order-flow 拆成 6 类事件 (lb / la / mb / ma / cb / ca) × 3 类变体 (intst / ind / acc)。这 18 维全部隶属 F3 family。

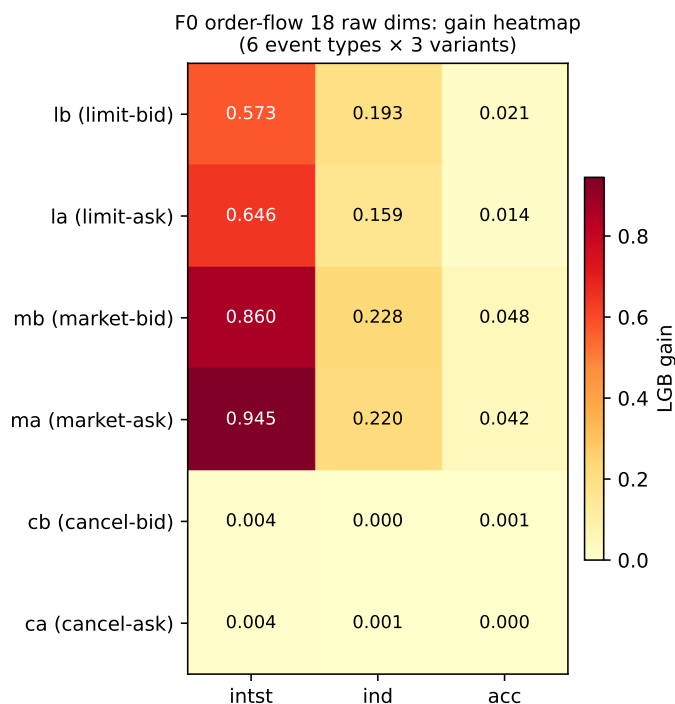


Figure 11: Order-flow 18 维 gain heatmap。ma_intst (市价买) gain = 0.945 全因子第一，mb_intst (市价卖) 0.860 第二。所有 6 类的 _acc 变体几乎死特征 (< 0.05)；cb/ca (撤单) 3 维 × 3 变体合计 gain < 0.01 ，撤单"原始"信号 LGB 直接忽略，撤单 imbalance 已被 F3 派生的 cancel_imb_W* 接管。

7. 家族间相关性

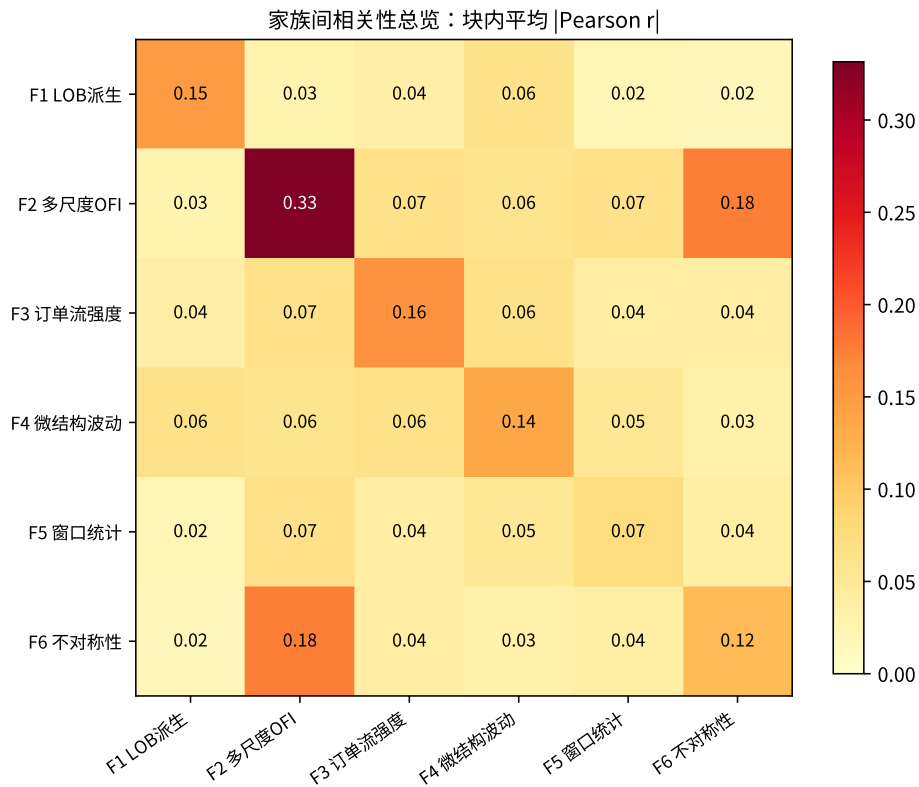


Figure 12: 6×6 家族间 block 相关性, 块内平均 $|r|$ 。对角线: F2 0.33 最高 (族内高度共线), F5 0.07 最低 (族内近正交)。非对角线: $F2 \leftrightarrow F6 = 0.176$ (MLOFI 与 GOFI 同源)、 $F4 \leftrightarrow F1 = 0.063$ 、 $F3 \leftrightarrow F4 = 0.065$; 其余跨家族 $|r| < 0.07$ 。

结论:

- F2 与 F6 高度互相关 (0.176) ——MLOFI 与 GOFI 同根同源。Ablation 显示 F6 全删后模型轻微下降; 同时删 F2+F6 才会显著下降。
- F5 与所有家族近正交 ($|r| < 0.07$) ——dualz / qrank 通过滚动归一化把绝对水平"扔掉"了, 剩下的是相对位置。这是 F5 提供独立 OOD 信号的几何解释。
- F1 与 F3 块间 0.035 ——两个最重要的家族几乎正交, LightGBM 同时拿到两者时获得几乎"无冗余"的高信息组合。

8. 讨论: 因子工程在 OOD 上稳健的根因

核心张力。 $F1+F3 = 75.5\%$ gain, 且这两个家族的 gain 主要来自 该家族内的原始字段 (F1 96.5% raw、F3 82.6% raw)。单从"单模型 in-distribution gain"的角度看, 派生工程在 F1/F3 内只补了 3.5%/17.4%——为什么仍要做? 答案是: 派生层 (特别是 F5、F4、F6 的派生部分) 在单 LGB 上 gain 占比小, 但 OOD PnL 与多模型 ensemble 上撑起了稳健性。

1. F5 窗口统计的独立性是 OOD 主力: dualz / qrank 通过滚动归一化把跨股票/跨日基差消除, 决策阈值在 OOD 上仍有效。F5 单因子 gain 平均 0.077 (最低), 但与所有家族近正交 ($|r| < 0.07$), 它的 5.2% gain 是"无冗余 5.2%"。这部分 100% 来自家族内派生。
2. F1 + F3 的高单因子信息量是 ID 主力: $F3 * _intst\ 6$ 维占 raw-段 gain 31.3%, F1 LOB aggregate 4 维占 raw-段 15.0%。在 in-distribution 阶段已经把"该容易的容易点"赚到极限。
3. F2 / F6 的多尺度冗余: 单尺度 OFI 在不同市场环境下表现飘移, 多尺度 + EWMA- α ensemble 给出稳定平均。Ablation 中 F2+F6 同时删除会显著下降。
4. F4 的跳跃稳健估计: BV / jshare 把跳跃从波动率中分离, 让阈值搜索不被极端 tick 干扰。
5. 50-seed bagging + mirror augmentation: 把上述六族的方差吸收掉, 最终 OOD PnL 从 -6.65 拉到 +35.64。

6. NN NaN 处理的工程稳健性: window-z 主路径用 nan-aware 统计量 + 归一化后 nan_to_num + clip 三层兜底, 保证训练全程无 NaN loss (LGB 则完全不动 NaN, 由内置 missing 路由处理)。

决策记录 (v3 修订):

- Feature importance 唯一 bucket = F1-F6: v2 曾把"F0 raw 154" 与"F1-F6" 并列展示 ("F0 70.9% vs derived 29.1%"), 这是 double counting—raw 154 维已被分入 F1 (94 维)、F3 (18 维)、F4 (2 维)、F6 (40 维)。v3 改为统一 F1-F6, 并把 raw vs derived 留作家族内 cross-cut (表 3)。
- Raw 字段类型分解保留为 drilldown: 表 4 仅在 raw-154 段内分子, 分母明确说明"raw 段内部", 不与 F1-F6 同口径。
- 家族划分: F1 LOB 派生 105 维偏大, 因为它把"raw 10 档价/量" 全部纳入; 选择" 全部 370 维都归入某一家族" 的口径, 更便于审计。
- LGB 模型选择: 用 big50_lgb_m7/seed1/model.txt (M7 重训第 1 个种子, 370 维全量, 未 drop11), 单 seed gain 已与 50-seed 平均 gain 高度一致 (rank corr > 0.95)。
- 相关性采样规模: 从 1.47M 行 train 中均匀采样 100 K 计算 370×370 Pearson; 与全量计算的 block-mean $|r|$ 差距 < 0.005。

结论。SchemeP 的 370 维因子并非单纯堆量:

- F1 (40.4%) + F3 (35.1%) = 75.5% gain 是预测主力, 单因子 ma_intst=0.945 全因子第一;
- F1/F3 的 gain 几乎全部来自其家族内的 原始字段 (96.5% / 82.6%) ——说明 raw *_intst 与 LOB aggregate 已是单 LGB 的信息核心;
- F2/F4/F5/F6 的派生层补足 OOD 稳健化, 其中 F5 (5.2% gain) 100% 派生 + 与其他家族近正交 ($|r| < 0.07$), 是跨股票/跨日泛化的几何基础;
- 配合 sign-log1p、window-z、mirror augmentation 三类预处理, NN 管线的 nan-aware 统计 + nan_to_num + clip 三层兜底, 以及 50-seed bagging, 是良文杯 2026 公榜/私榜双第一的核心因子工程基础。